PROJECT PRESENTATION

SkillMatch

Skill-based job & company recommendation system

A full-stack web application that helps people discover roles aligned with their skills, experience, and career preferences.

PROFILE | DISCOVER | GET MATCHED

Find work that fits.

TERM PROJECT • SOFTWARE ENGINEERING

The opportunity

Why SkillMatch is useful

The challenge

- Job seekers search across many listings and company pages.
- A role title alone does not show how well a candidate fits.
- People need clear feedback on skills they already match and skills to build.

The project goal

- Build one place to create a career profile and explore opportunities.
- Rank jobs with a transparent, configurable matching score.
- Explain matches and skill gaps so users can make informed next steps.

Project outcome: useful job discovery with understandable, profile-based recommendations.

Who uses SkillMatch?

Two user roles share the same opportunity catalogue

Job seeker

- Register and sign in securely
- Build a profile with skills, experience, education, preferences, projects, and certifications
- Search and filter jobs and companies
- Review personalized match reasons and missing skills
- Save interesting jobs and companies

Administrator

- Manage companies and job listings
- Review basic marketplace overview
- Adjust relative recommendation weights
- Keep catalogue data current for job seekers

Optional profile feature: upload a PDF or Word resume (up to 5 MB).

A simple user journey

From career information to an explainable opportunity match

01 — 02 — 03 — 04 — 05

Create account

Register and sign in

Build profile

Add skills, experience & preferences

Explore roles

Search jobs and companies

Understand fit

See match score, reasons & skill gaps

Save & act

Keep opportunities for later

Profile data is used to personalize matching; users can also browse and apply their own filters.

How recommendations work

A weighted score balances several candidate-to-role factors



Required skills	**50%**	**Experience**	**15%**
Education	**10%**	**Preferred role**	**10%**
Industry	**5%**	**Location**	**5%**
Certifications & projects	**5%**		

The skill factor combines required-skill coverage (85%) with text similarity (15%).

Explainable output

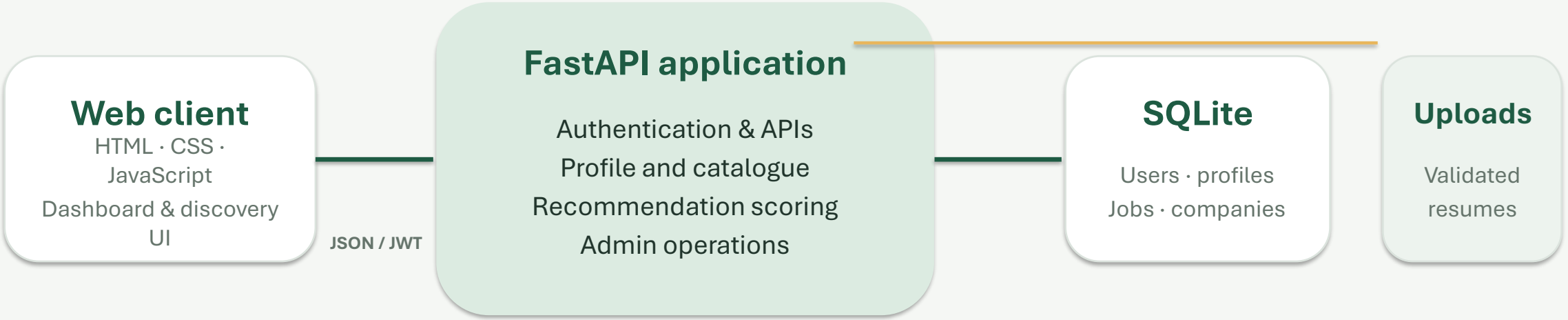
For each role, SkillMatch can show:

- Overall match score
- Skills matched
- Skills still missing
- Factor scores and plain-language reasons

Weights are normalized and admin-adjustable.

System architecture

A browser client communicates with a Python API and SQLite data store



Local development: frontend http://127.0.0.1:5500 · API http://127.0.0.1:8000 · API docs /docs

Backend: FastAPI + Pydantic · Persistence: SQLite · Recommendation logic: standalone Python module

Core data and API capabilities

The application organizes candidate information and opportunity catalogue data

Main data entities

- Users and authentication roles
- Career profiles, skills, education, certifications, and projects
- Companies, jobs, and required skills
- Saved jobs/companies and recommendation history

Main API areas

- Authentication and current-user endpoints
- Profile read/update and resume upload
- Job and company search with filters
- Recommendations, saved items, dashboard, and admin management

Interactive API reference: /docs · Request and response schemas are documented in OpenAPI.

Security and configuration management

Protecting users and controlling project change

Security controls in the project

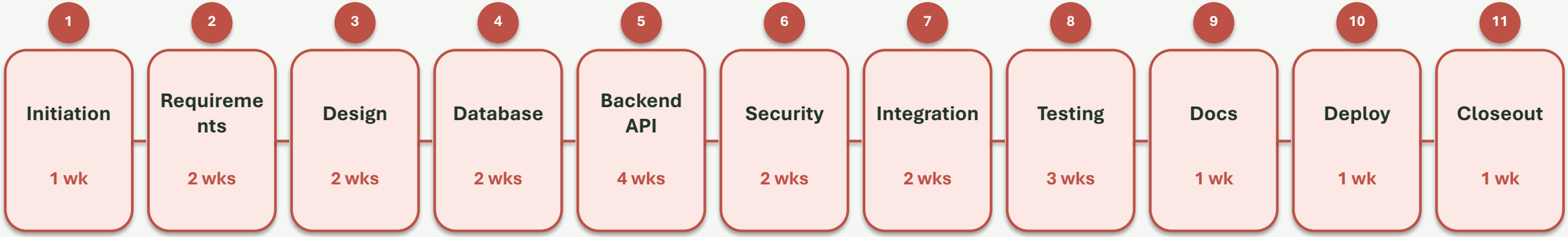
- Passwords hashed with scrypt
- Signed JWT authentication and role-based admin access
- Validated resume types and 5 MB upload limit
- Input validation for API request models
- CORS allow-list for frontend origins
- Keep deployment secrets out of source control

SCM objects to manage

- Backend, recommender, and frontend source files
- Database schema, dependency manifest, and safe config examples
- Tests, documentation, and release notes
- Change requests, defects, and review history
- Protect the database, real user data, and uploaded resumes

Project plan at a glance

Estimated Critical Path Method schedule; durations are planning assumptions



Estimated critical path: Initiation → Requirements → Design → Database → Backend → Security → Integration → Testing → Documentation → Deployment → Closeout

Parallel work: frontend development (4 weeks) and recommendation engine (3 weeks) are planned alongside backend work. Estimated total: 21 weeks.

CPM baseline from the project plan; actual delivery time can change with scope, resources, and test findings.

Monitoring progress with Earned Value

Example figures only — replace with actual budget, progress, and cost records

PV

₹8,52,000

Planned value by week 10

EV

₹8,10,000

Budgeted value of work completed

AC

₹8,90,000

Actual spending by week 10

Schedule & cost signals

- SV = EV − PV = −₹42,000 → behind schedule
- CV = EV − AC = −₹80,000 → over budget

Performance indexes

- SPI = EV ÷ PV = 0.95
- CPI = EV ÷ AC = 0.91
- Illustrative EAC ≈ ₹13,18,519

Interpretation: investigate schedule variance and cost efficiency; use actual time sheets, invoices, and verified task completion.

Validation and expected value

Where the project is today and what it delivers

Project validation

- Automated tests cover auth, profiles, recommendations, search, saved items, and admin access
- The current test suite was verified with 11 passing tests
- Local app and API docs are available through separate frontend and backend servers

Value delivered

- A unified place to explore jobs and companies
- Recommendations that reveal match evidence and skill gaps
- Configurable scoring and admin-managed catalogue
- A clear foundation for future improvements

Thank you • Questions?

Demo URLs: http://127.0.0.1:5500 | API documentation: http://127.0.0.1:8000/docs